Lab 0: Top Level

Khoi Huynh, Joy Mossia, Luc Mwizerwa, Jace Rausch, Jay Singh
Department of Computer Science
University of Oklahoma

***Abstract*—This lab builds a discrete-time queueing simulation platform and uses it to study how demand, waiting capacity, service capacity, and topology affect latency and loss. Synthetic Poisson workloads feed configurable multi-server FIFO queues whose service is expressed as `Sub-X` work reduction; per-step metrics and event logs are analyzed with USE measurements and response-time percentiles. The platform is applied to an M/D/1-style multi-stage car wash and to exit-directed traffic on sampled SNAP `roadNet-TX` subgraphs at three scales. Across both studies, expanding buffers mainly redistributes waiting and rejection rather than raising throughput, while adding servers at the true bottleneck reduces delay and loss. Network size alone does not determine congestion: single-exit and interior merge structures dominate performance even when many exits exist.**

I. Introduction

A. Objectives: What We Are Trying to Do

This lab builds and applies a complete discrete-time queueing simulation platform, from raw workload generation through empirical analysis and multi-scale modeling. The platform is organized into six objectives. Objectives A through C establish the toolchain: Objective A generates synthetic Poisson-arrival workloads; Objective B implements a configurable multi-server FIFO queue simulator supporting arbitrary service-reduction operations (`Sub-X`); and Objective C provides nine analysis and visualization tools that measure utilization, saturation, and errors following the USE method [1], along with response-time percentiles (P10, P50, P90). Objective E extends the single-queue engine into a general queueing network, routing jobs across multiple bounded queues connected by a directed graph.

Building on this infrastructure, Objectives D and F apply the simulator to two real-world scenarios. Objective D models a multi-stage automated car wash with five sequential service phases (pre-rinse, soap, scrub, rinse, dry) totaling $S{=}34$ timesteps, as an M/D/1-style single-queue system, then runs 60 controlled experiments across three demand levels and six resource-scaling variants ($\pm10\%$ to $\pm100\%$ in bay count and waiting capacity). Objective F maps exit-directed vehicle flow onto a sampled subgraph of the Stanford SNAP `roadNet-TX` dataset [2], treating each intersection as a bounded FIFO queue and each road as a transfer link. The study is executed across three scales: SMALL (20 queues, 1 exit sink, $\sim$2,000 vehicles) and MEDIUM (200 queues, 10 exit sinks, $\sim$20,000 vehicles) across 27-run experiment matrices, and LARGE (2,000 queues, 62 exit sinks, $\sim$200,000 vehicles) across a 7-configuration resource sensitivity study.

B. Motivation: Why It Is Important

Resource contention and queueing delay are defining performance challenges in distributed and operating systems. As an open queueing system approaches its service capacity, mean latency grows non-linearly; near saturation, even small increases in offered load produce disproportionate increases in waiting time and rejection rate [3]. This behavior makes capacity planning, bottleneck identification, and resource-allocation decisions difficult to reason about analytically, particularly when service is multi-stage and topology is graph-structured.

Discrete-event simulation with structured telemetry addresses this gap directly. The USE method provides a systematic lens: utilization quantifies how fully servers are occupied, saturation reveals queue backpressure, and error counts capture load-induced loss, together exposing the full performance envelope before and after a saturation boundary [1]. The car-wash study demonstrates that expanding waiting capacity primarily shifts *which* arrivals are rejected rather than increasing throughput, a result that generalizes to buffer-sizing decisions in real service pipelines. The road-network study demonstrates how local queue overflow propagates spatially through a connected topology, causing high-distance nodes to shed nearly one-third of vehicles under overload, a pattern relevant to any network of communicating service nodes. Together, these case studies validate the simulation platform against analytically expected behavior and provide empirically grounded guidance for resource-scaling decisions in systems where saturation effects are otherwise hard to anticipate.

II. Theory

A. The Queueing Problem and Why It Matters

A queue forms whenever work arrives at a system faster than it can be handled immediately. The arriving work might be a person waiting at a checkout, a car waiting for a washing bay at the carwash, or a request waiting for a computer server. Although these examples look different, they share the same basic process: jobs arrive, wait if the required resource is occupied, receive service, and eventually leave. Queueing theory is the study of this process and of how waiting, service capacity, and arrival patterns affect performance. It provides a simpler mathematical model of a system so that different designs can be studied before resources are added or changed. [4]

The main difficulty is that the amount of incoming work is not always predictable. Two systems can receive the same average number of jobs but behave differently if one receives

jobs evenly and the other receives several jobs at once. A server may be able to handle the average demand and still experience a temporary backlog. As demand approaches its processing capacity, even a small increase in arrivals can produce a much larger increase in waiting time. If the waiting area is limited, some jobs may not be admitted at all. These effects matter because they influence both the experience of an individual customer and the amount of useful work completed by the system. [4]

For this lab, the problem is to represent these interactions in a configurable simulator and use its output to explain when a system has enough resources, when it becomes congested, and which resource changes address the cause of that congestion. We consider both the Objective B Simple Queue Simulator and the Objective E Multi Queue Simulator.

B. Basic Components of a Queueing System

The workload format uses the exact columns `Job Id`, `arrival time`, and `Tasks`. Each row represents one job. `Job Id` is the unique job name, `arrival time` is the timestep when the job enters the system, and `Tasks` is the ordered list of work for the job. The repository writes an individual task in the form `<task-id>: W <positive work amount>`. The given PowerPoint represents the same form as `<tid>: W N`, where `N` is a positive number. A job is finished when `Tasks` is empty.

The Queue Configuration uses the following exact column names:

- `Queue sys ID`: the queue-system name used in metrics and events.
- `Max Queue Size`: the maximum number of jobs waiting for a server.
- `# of servers`: the number of jobs that may be in service at once.
- `Server Function`: the operation used to modify the current task, such as `Sub-X`, a chain of functions, or a repeating chain.
- `Server Function Rate`: the number of timesteps used by each server-function step.
- `source`: the source workload filename.
- `sink`: the completed-job workload filename.
- `Edges in`: the incoming edge names.
- `Edges out`: the outgoing edge names.

The Simple Queue Simulator describes its basic flow as a `source workload` feeding a FIFO waiting queue, which then feeds the servers. Jobs leaving the servers can move to a `sink / outgoing edge`. `Max Queue Size` counts only jobs waiting for a server and does not include jobs already inside servers.

Let $Q_{\max}$ represent `Max Queue Size` and let c represent `# of servers`. These are two different resources. For example, a configuration with $Q_{\max} = 20$ and $c = 1$ can contain up to 20 waiting jobs plus one job in service. Increasing `Max Queue Size` allows more jobs to wait, but it does not make a server process work faster. Increasing `# of servers` changes the amount of work that can be processed at the same time.

C. Random Arrivals and Synthetic Workloads

To study a queue, we must first describe how often jobs arrive. The arrival rate, written as λ (lambda), is the expected number of new jobs per unit of time. A useful model for independent, random arrivals is the Poisson distribution. If A_t is the number of jobs arriving during timestep t, a Poisson workload assumes

$$P(A_t = k) = \frac{e^{-\lambda}\lambda^k}{k!}, \qquad k = 0, 1, 2, \ldots,$$

where $P(A_t = k)$ is the probability of exactly k arrivals in one timestep, e is a mathematical constant, and $k!$ is the factorial of k. Under this model, the expected number of arrivals per timestep is $E[A_t] = \lambda$, and counts in separate timesteps are assumed to be independent. Therefore, a small average rate does not mean that exactly one job arrives at regular intervals: one timestep may have no arrivals, while another may have multiple arrivals. [5]

The Workload Generator supports the distributions `uniform`, `normal`, `geometric`, and `poisson`. It also supports fixed task work or task work drawn from the same distribution. This makes it possible to investigate different demand patterns using synthetic workloads rather than requiring measurements from a real business. The completed car-wash study uses Poisson arrival counts to represent random customer demand and fixed service requirements so that the effects of arrival variability and resource changes can be studied separately. The completed multi-queue traffic study also uses Poisson arrival counts, while the amount of work assigned to each vehicle is determined by its distance from an exit in the generated network model.

D. Service Time, Offered Load, and Congestion

The service time, S, is how long a server is occupied by one job. In the simulator, the `Server Function` modifies the remaining work of the job's current task. For example, `Sub-1` removes one unit of work when the function is applied. A trailing `*`, as in `Sub-1*`, repeats the function chain while tasks remain. The configured `Server Function Rate` gives the number of timesteps used by each server-function step. Multiple operations may therefore be required before a job is completed.

For a simple system in which every job requires the same S timesteps, one server can complete at most approximately $1/S$ jobs per timestep when continuously busy. With c identical servers, the nominal processing capacity is

$$C_{\text{nominal}} = \frac{c}{S} \quad \text{jobs per timestep.}$$

A useful comparison between demand and capacity is the offered load,

$$\rho_{\text{offered}} = \frac{\lambda}{C_{\text{nominal}}} = \frac{\lambda S}{c}.$$

This value compares how much work is offered to how much the servers could process. It is not itself a measurement of actual server utilization. If $\rho_{\text{offered}} < 1$, average incoming demand is below nominal processing capacity, although random arrival bursts can still cause waiting. As the ratio approaches one, the servers have less spare capacity to clear a backlog. If $\rho_{\text{offered}} > 1$ for a sustained period, incoming work exceeds the nominal processing rate. With unlimited waiting space, the backlog can continue to grow. With the lab's finite `Max Queue Size`, the waiting queue can fill and new jobs can receive `ERROR QUEUE-FULL`. [5]

A common introductory theoretical model is called an M/M/1 queue. Its first two letters describe memory-less arrival and service processes, and the final number indicates one server. For its idealized unlimited-waiting-space version, the long-term probability of observing n jobs has the form

$$\pi_n = (1-\rho)\rho^n, \qquad \rho = \frac{\lambda}{\mu} < 1,$$

where μ is the average service rate. This result illustrates why the relationship between arrivals and service is important. [5]

The completed car-wash model should not be treated as an exact M/M/1 system. The final lab describes it as a discrete-time model with Poisson arrival counts, deterministic service, and finite waiting capacity. The one-bay baseline resembles a finite-capacity M/D/1 queue, except that arrivals are grouped into integer timesteps rather than occurring at continuous times. The two-bay experiment uses the same deterministic service model with two servers. [5]

E. Simple Queue Simulator and Multi Queue Simulator

The Objective B Simple Queue Simulator models one FIFO waiting queue with one or more servers. Its documented flow is a `source workload` entering a FIFO waiting queue, followed by the servers, and then a `sink / outgoing edge`. A completed job with a `sink` can be written to the sink workload. If a job is routed through an outgoing connection, the configured name appears under `Edges out`.

The Objective E Multi Queue Simulator extends this model to multiple connected queue systems. A job that finishes service at one queue may be transferred through an outgoing edge and later receive service at another queue. When more than one outgoing edge is available, the simulator chooses among the permitted outgoing edges using round-robin routing. The general queueing-theory idea is a network of queues, where completion at one queue may cause the job to join another queue rather than leave the entire system. [5]

Objective F applies the Multi Queue Simulator to an exit-directed synthetic traffic model using sampled road connectivity from SNAP roadNet-TX. A vehicle is represented as a job, an intersection is represented as a bounded FIFO queue, and servers represent abstract simultaneous intersection-processing capacity. Edges represent allowed transfers between intersections. Traversing an edge does not add a separate travel-time delay in the simulator.

For a sampled undirected road graph, the model selects exits from boundary nodes and computes each node's shortest distance d to a selected exit. Permitted connections are then directed from a node at distance d toward a node at distance $d-1$. Exit nodes contain a sink and have no outgoing queue edges. Because every permitted transfer decreases the distance by one, a vehicle continually moves closer to an exit and cannot revisit a queue in this model.

A vehicle entering at distance d receives one task with work `W (d+1)`. Each queue uses `Sub-1` without a trailing `*`. Each serviced visit therefore removes one work unit, and the final work unit is processed at the exit. For example, a vehicle entering at distance 2 begins with `W 3`, reaches the next queue with `W 2`, reaches the exit with `W 1`, and is then completed. With no waiting and a `Server Function Rate` of 1, a vehicle entering at distance d requires $d+1$ serviced queue visits.

The completed Objective F studies use three scales. SMALL contains 20 queues, MEDIUM contains 200 queues, and LARGE contains 2,000 queues. The baseline configuration at all three scales uses four servers and 20 waiting spaces per queue. SMALL has one selected exit, MEDIUM has 10 exits, and LARGE has 62 exits. These are synthetic exit-directed traffic studies using sampled road connectivity. They do not claim to reproduce actual traffic volumes, traffic signals, lane capacities, travel times along roads, turning behavior, or driver route choice.

F. Measuring Queue Performance

The Simple Queue Simulator writes Queue Metrics to `qmet.csv`. The exact header is

```
time-step,queue-sys-id,num-jobs-in-queue,
num-jobs-in-service,num-of-errors
```

The fields have the following meanings. `time-step` is the current simulation time, `queue-sys-id` identifies the queue, `num-jobs-in-queue` is the number of waiting jobs, `num-jobs-in-service` is the number of busy servers, and `num-of-errors` is the cumulative number of errors.

The given PowerPoint groups the main resource measurements as Utilization, Saturation, and Errors, or USE. Utilization describes how much the servers are being used, saturation describes how many jobs are waiting in the queue, and errors record problems that occur while jobs move through the system. The queue metrics file provides the values `num-jobs-in-queue`, `num-jobs-in-service`, and `num-of-errors`, which are used to calculate these measurements.

The Objective C tool `compute-use` calculates USE measurements over windows of 10, 100, and 1000 timesteps. It writes the exact columns `Utilization(10)`, `Saturation(10)`, `Errors(10)`, `Utilization(100)`, `Saturation(100)`, `Errors(100)`, `Utilization(1000)`, `Saturation(1000)`, and `Errors(1000)`.

There is an important distinction between the general meaning of utilization and the value produced by the delivered `compute-use` implementation. The implementation checks whether at least one server is busy during each timestep and averages that value over the selected window. The result is then rounded, so `Utilization(W)` acts as an activity indicator rather than an exact percentage of all available server capacity. `Saturation(W)` represents the rolling average number of jobs waiting in the queue. In the completed analyses, the error measurements represent the number of new errors occurring during the selected window after adapting the cumulative error input. They are not error rates per unit time even though the plotting tool uses the title `Error Rate`.

The Objective C tool `derive-event-metrics` writes the exact columns

```
job-id,queue-sys-id,arrival-time,
time-in-queue,time-in-service,
response-time,turn-around-time
```

For this tool, `response-time` means the time from arrival until the first `ENTERS-SERVER` event for that queue visit. `turn-around-time` means the time from arrival until `DEPART-VIA` or `ERROR`. The completed analyses therefore use explicit terms such as waiting time, service time, and total time when necessary to avoid confusing the different uses of response time.

For a completed single-queue job j, let a_j be its original arrival time, s_j its first `ENTERS-SERVER` time, x_j its final `EXITS-SERVER` time, and d_j its final `DEPART-VIA` time. Then

$$\begin{aligned} W_{q,j} &= s_j - a_j \quad \text{(waiting time)}, \\ S_j &= x_j - s_j \quad \text{(service time)}, \\ T_j &= d_j - a_j \quad \text{(total time / turnaround)}. \end{aligned}$$

The Objective C tool `derive-sys-metrics` writes the exact columns

```
time-step,Avg-response-time,
median-response-time,10-response-time,
90-response-time,throughput
```

In these system metrics, response time refers to the full arrival-to-completion interval rather than arrival-to-first-service. The `throughput` field records the number of jobs exiting at a particular `time-step`.

For an observation window of H timesteps, a throughput rate can be written as

$$X_H = \frac{D_H}{H} \quad \text{jobs per timestep},$$

where D_H is the number of completed jobs during that interval. A job still waiting or receiving service at the observation cutoff does not count as a completion before that cutoff, even if it later finishes while the system is draining.

For the Multi Queue Simulator, an internal departure from one queue to another is not a completed vehicle and therefore does not count as whole-network throughput. Whole-network throughput is based on final departures through the exit sinks. Similarly, whole-network response time is measured from a vehicle's original external arrival until its final sink departure. This differs from an individual queue visit, which has its own arrival, service, and departure events.

The checked Objective E event log records external `ARRIVE-VIA` events but does not record a new `ARRIVE-VIA` event when a vehicle is transferred from one queue to another. Objective F therefore preserves the original event log and creates a separate analysis copy. The analysis copy inserts a same-timestep downstream `ARRIVE-VIA` after each validated inter-queue transfer so that the Objective C tools can measure every queue visit. The raw simulator output remains unchanged.

The completed analyses also report mean, median, P10, and P90 values for completed-job timing measurements. Looking beyond an average is useful because a smaller group of jobs may experience much longer delays than most completed jobs.

As an additional consistency check, the completed single-queue analysis uses Little's Law for consistently defined system boundaries and observations:

$$L = \lambda_{\text{accepted}} W.$$

Here, L is average occupancy, $\lambda_{\text{accepted}}$ is the accepted-job arrival rate, and W is average total time in the system. Occupancy includes both waiting and in-service cars. Immediately rejected arrivals are not included as accepted demand. For a finite run, the same relationship can be checked by comparing summed customer total times with queue and server occupancy over matching time boundaries.

G. Applying the Theory to the Car-Wash Study

The completed Objective D study models a car wash with one shared waiting line feeding identical wash bays. One car is represented as one job, the waiting driveway or lot is represented by the bounded FIFO queue `q0`, and one wash bay is represented by one server. A car that arrives when the waiting lot is full receives `ERROR QUEUE-FULL`. A completed wash leaves through the `sink` with no remaining tasks.

Each car must complete five ordered tasks in the same bay, those being Pre-Rinse, Soap, Scrub, Rinse, and Dry. The assumed amounts of work are 5, 8, 10, 6, and 5 units, for a total of

$$S = 5 + 8 + 10 + 6 + 5 = 34 \text{ timesteps}.$$

The car-wash configuration uses `Sub-1*` with `Server Function Rate` equal to 1. One timestep represents 10 seconds, so a complete wash requires 340 seconds, or 5 minutes and 40 seconds. These durations are assumptions for the synthetic study and are not measurements from an operating car wash.

The baseline configuration has `Max Queue Size` equal to 20 and `# of servers` equal to 1. Its nominal processing capacity is therefore

$$C_{\text{nominal}} = \frac{1}{34} \text{ cars/timestep} \approx 10.59 \text{ cars/hour},$$

using 360 ten-second timesteps per hour.

The experiment uses three demand scenarios with Poisson arrival parameters of $\lambda = 0.010$, 0.025, and 0.040 cars per timestep. The final lab labels these scenarios Light, Near capacity, and Over capacity. They correspond to expected offered demand of 3.6, 9.0, and 14.4 cars per hour. With one bay, the offered-load calculation gives 34%, 85%, and 136%, respectively.

The model assumes that all cars receive the same wash package and service requirement. Cars do not abandon the queue, rejected cars do not retry, and payment, entering or leaving the bay, equipment failure, staffing, and water constraints are not modeled separately. A car remains assigned to the same bay until all tasks are complete, and service is not preempted.

The waiting-space sensitivity study keeps the bays, wash duration, and workload fixed while changing the waiting spaces from the baseline of 20 to 25, 30, 40, 18, 16, and 14. These correspond to $+25\%$, $+50\%$, $+100\%$, -10%, -20%, and -30%. A separate comparison changes the number of bays from one to two while keeping 20 waiting spaces. The same workload is reused for matching resource variants so that the effect of changing the resource can be compared directly.

H. Applying the Theory to the Multi-Queue Traffic Study

The completed Objective F study applies the Multi Queue Simulator at three scales. SMALL uses 20 intersections, MEDIUM uses 200 intersections, and LARGE uses 2,000 intersections. Vehicles are generated using Poisson arrival counts, assigned to ingress nodes, and routed toward selected boundary exits. The baseline at every scale uses four servers and 20 waiting spaces per queue.

The network topology changes the meaning of capacity compared with the single car-wash queue. In the car-wash model, the primary processing resource is the number of wash bays. In the traffic model, congestion can occur at different intersections depending on how several routes merge. Increasing the total number of servers can reduce waiting and loss, but adding capacity at queues that are not bottlenecks may have little effect on the queues that actually limit flow.

For SMALL, the network contains 20 queues, 19 ingress nodes, one exit, and a maximum exit distance of 8. The stressed seed-42 workload contains 2,067 offered vehicles. The baseline has 80 total servers, completes 2,013 vehicles, drops 54 vehicles, and has a mean whole-network response time of 9.723 timesteps. Increasing total server capacity by 25% to 100 servers eliminates all drops for this workload and lowers mean response time to 5.961 timesteps. Reducing capacity by 30% to 56 servers produces 485 dropped vehicles, a 23.46% loss rate, and a mean response time of 24.200 timesteps among completed vehicles.

For MEDIUM, the network contains 200 queues, 190 ingress nodes, 10 exits, and a maximum exit distance of 14. The stressed seed-42 workload contains 19,879 offered vehicles. The baseline has 800 total servers, completes 19,878 vehicles, drops one vehicle, and has a mean whole-network response time of 8.23 timesteps. A 25% server increase to 1,000 total servers eliminates the remaining loss and lowers mean response time to 7.39 timesteps. A 30% reduction to 560 servers produces 969 dropped vehicles, a 4.87% loss rate, and raises mean response time to 15.56 timesteps.

For LARGE, the network contains 2,000 queues, 1,938 ingress nodes, 62 exits, and a maximum exit distance of 45. The completed sensitivity study uses a stressed workload containing 200,372 offered vehicles. The baseline has 8,000 total servers, completes 174,105 vehicles, drops 26,267 vehicles, and has a mean whole-network response time of 22.04 timesteps. Doubling the number of servers to 16,000 lowers the loss rate from 13.11% to 4.53% and lowers mean response time to 19.11 timesteps. Reducing the server pool by 30% to 5,600 raises the loss rate to 15.76% and raises mean response time to 26.41 timesteps.

The three scales demonstrate that network size alone does not determine congestion. SMALL sends all routes toward one exit, so that exit naturally limits the network's maximum departure rate. MEDIUM distributes traffic across 10 exits and handles the stressed baseline workload with almost no loss. LARGE has 62 exits, but many ingress flows merge at shared interior queues before reaching those exits. In the LARGE model, these interior merge queues can therefore become bottlenecks even though the network has many possible exits.

I. What the Theory Helps Explain

The completed experiments show the distinction between waiting capacity, processing capacity, demand, and network topology. In the car-wash model, increasing waiting space allows more cars to remain in the system but does not increase the processing rate of the wash bay. Increasing the number of bays directly increases service capacity. In the Multi Queue Simulator, the same principle applies at each individual queue, but the location of a queue within the network also determines how much traffic reaches it.

In the long Over capacity one-bay car-wash experiments, average waiting time among completed cars was approximately 97–99 minutes, and roughly 22% of offered cars were rejected. Doubling the waiting space admitted additional cars but increased the average wait among completed cars to approximately 192–200 minutes without increasing throughput during the common arrival window. Adding a second bay instead reduced average waiting to approximately 2–2.4 minutes and produced no rejections in those tested Over capacity workloads.

The completed network experiments show that topology can create different bottlenecks even when the same basic queue rules are used. At stressed baseline capacity, the SMALL, MEDIUM, and LARGE seed-42 studies had loss rates of approximately 2.61%, 0.01%, and 13.11%, respectively. Their mean whole-network response times were approximately 9.72, 8.23, and 22.04 timesteps. SMALL is strongly affected by its single-exit structure, MEDIUM benefits from 10 distributed exits, and LARGE experiences important congestion at interior merge queues despite having 62 exits.

The resource-sensitivity results also show that additional capacity does not have the same effect at every scale. In SMALL, a 25% increase in servers is enough to eliminate all loss in the stressed seed-42 workload. MEDIUM is already nearly lossless at its stressed baseline and also becomes lossless with a 25% increase. LARGE remains lossy even after server capacity is doubled, although the loss rate falls from 13.11% to 4.53%. This shows that the effect of adding resources depends on where congestion forms in the network and how traffic converges through its topology.

Together, the theory and measurements show why `num-jobs-in-queue`, `num-jobs-in-service`, errors, waiting time, total response time, throughput, and topology must be interpreted together. A resource change can improve one measurement while leaving another unchanged or making it worse. The simulator provides a consistent way to observe these tradeoffs in both the Simple Queue Simulator and the Multi Queue Simulator.

III. Mechanisms

The lab is organized as a fixed pipeline (workloads $\rightarrow$ discrete-time simulation $\rightarrow$ telemetry $\rightarrow$ analysis) so that every modelling study reuses the same artifacts and interfaces. Objective A synthesizes seeded Poisson (and related) arrival counts per integer timestep into a shared workload CSV of the form `Job Id,arrival time,Tasks`, where each task carries a work quantum `W` w. Objective B then advances a configurable multi-server FIFO system one timestep at a time in a fixed order: service advance, FIFO dispatch to idle servers, arrival admission and routing, then metrics emission. Service is expressed as a `Sub-X` reduction on the head task (optionally chained or starred to repeat until the job empties): after every `Server Function Rate` steps, the head work is decreased by X or removed when $W \leq X$. Bounded waiting capacity produces `QUEUE-FULL` rejections; completed jobs exit to a sink or round-robin over declared outgoing edges. Per-step occupancy and cumulative errors are written to `qmet.csv`, while a fine-grained event log records arrivals, server entries/exits, and departures for later reconstruction of response times.

Objective C turns those traces into USE and latency views without changing the simulator: rolling windows of size 10, 100, and 1000 estimate utilization (fraction of steps with at least one busy server), saturation (mean queue length), and error mass; system metrics derive nearest-rank P10/P50/P90 response times and throughput at each sink-exit timestep. Objectives D and F specialize this stack. The car-wash study maps five deterministic stages ($5{+}8{+}10{+}6{+}5{=}S{=}34$) onto a single bay configured as `Sub-1*`, yielding an M/D/1-style finite-buffer system whose 60-run matrix crosses light/near/over demand with lot-size and bay-count variants on paired Poisson workloads. Objective E lifts the same tick loop onto a directed graph of queues: SNAP (or Matrix Market) edge lists are compiled into per-node queue configurations; all queues advance synchronously, and same-tick transfers are buffered so routing is order-independent. Outgoing edges are selected round-robin; optional acyclic projection prevents perpetual circulation on undirected road graphs. Objective F instantiates that network engine on sampled `roadNet-TX` subgraphs at SMALL/MEDIUM/LARGE scale, treating intersections as bounded FIFO queues and roads as transfer links, then reuses the Objective C USE and percentile tools across the 27-run demand–seed–server matrix.

IV. Analysis and Results

Objective D: Single-Queue Analysis and Results

Mapping. We model a car wash. A car is a job, the waiting lot is the queue and a wash bay is a server. Every car takes 34 timesteps, so one bay washes about 10.6 cars an hour. We ran three demand levels, 3.6, 9.0 and 14.4 cars an hour, with one bay and 20 waiting spots.

Results. At 3.6 and 9.0 cars an hour the lot never fills and nobody is turned away. At 14.4 the bay cannot keep up. The lot fills in the first hour, cars wait 99 minutes on average and 22% of cars are turned away.

Sensitivity. At 14.4 cars an hour we changed the lot from 14 spots to 40. Throughput stayed at 10.5 cars an hour every time. A bigger lot just made a longer line, 200 minutes at 40 spots. Adding a second bay instead brought the wait down to 2 minutes with nobody turned away.

Takeaways. More waiting room does not fix a slow server. Another server does.

Objective E: Multi-Queue Analysis and Results

Mapping. We model Internet traffic between autonomous systems (AS) as a network of queues: each AS is a queue with its own servers, each link is an edge jobs follow after service, and a job is a unit of traffic entering at a source AS and leaving at a sink AS. The topology is the real SNAP AS-733 graph, turned into a configuration file by `create-config-from-network`. The graph is undirected, so we keep only edges from lower to higher node ids (`--acyclic`) to avoid infinite loops. Queues with no incoming edge get a source, and queues with no outgoing edge get a sink. The metrics are throughput, mean response time, drops (`QUEUE-FULL`), and USE per queue. Workloads come from the Obj-A generator: one job per tick per source queue, each with one task of work 10 (fixed seed). Every queue uses `Sub-10*` with a waiting room of 10 jobs.

Results by scale. With one server per queue, every run was badly overloaded: the entry queue serves one job per 10 ticks while one job arrives per tick, so only about 10% of jobs got through (210 of 2,000 and 2,010 of 20,000) and the rest were dropped. We therefore use 10 servers per queue as the baseline, which matches the arrival rate.

Sensitivity. We change the number of servers at every queue (10 is the baseline). Throughput is the share of offered jobs that reached a sink; mean response time is for the medium network, over completed jobs.

Scale	Queues	Src	Offered	Sink	Dropped
Small (10s)	20	1	2,000	2,000	0
Medium (100s)	200	1	20,000	20,000	0
Large (1000s)	2,000	18	900,000	900,000	0

TABLE I
BASELINE MULTI-QUEUE THROUGHPUT BY SCALE.

Servers	Change	Throughput (S / M / L)	Dropped (M)	Mean response (M)
7	-30%	70.5 / 70.1 / 70.0%	5,990	24.3
8	-20%	80.5 / 80.1 / 80.0%	3,990	22.5
9	-10%	90.5 / 90.1 / 90.0%	1,990	21.1
10	baseline	100 / 100 / 100%	0	10.0
12–20	$+20\%$ to $+100\%$	100 / 100 / 100%	0	10.0

Discussion. Throughput falls in a straight line with lost servers (each server removed costs about 10% of the traffic), and the percentages are almost identical at all three scales. So the limit is set by the entry queue, not by the size of the network. Response time, however, jumps from 10.0 (pure service time) to about 21 after only -10%, because the waiting room fills. Adding servers beyond the baseline gave no gain: nothing was dropped and response time was already at its minimum. The baseline is deterministic (exactly one arrival per tick), so it runs at full utilization without queueing; with random arrivals it would need headroom.

Verification. The simulator has 96 automated checks run under Valgrind, and `pretty-print-events` has 34 tests with 100% coverage. We found and fixed a bug where several sink queues overwrote one shared sink file (6 of 13 lines kept).

Takeaways. Fix the slowest queue first, never run at 100% utilization, and size buffers for bursts rather than average load.

OBJECTIVE F: MULTI-QUEUE ANALYSIS AND RESULTS

Mapping. We model traffic on the Texas road network. Every intersection is a queue with four servers and 20 spots, every road is an edge and every vehicle is a job. SMALL has 20 intersections and one exit, MEDIUM has 200 and 10 exits, LARGE has 2,000 and 62 exits.

Results by scale. At baseline SMALL dropped 2.6% of vehicles, MEDIUM dropped almost none and LARGE dropped 13%. SMALL sends everything to one exit, so that exit limits the network. MEDIUM spreads traffic over ten exits. LARGE has many exits but roads merge at the same inside intersections first, and only 34 of the 2,000 intersections ever have a line.

Sensitivity. We changed servers from -30% to $+100\%$ at each size. At SMALL and MEDIUM, $+25\%$ stopped all drops. At LARGE, doubling the servers only brought drops from 13% to 4.5%, because most of the new servers went to intersections that were never busy.

Takeaways. Find the queue that is holding everything up and add capacity there. Adding it everywhere is mostly wasted.

V. SUMMARY

Our group developed a customizable discrete-time queueing simulation which shows how workload, service capacity, queue size, and network structure affect system performance. The single-queue car wash example shows that increasing waiting space mainly increases how long jobs remain in the system, wile adding service capacity directly reduces waiting and dropped jobs. The multi-queue examples showed that congestion depends heavily on where bottlenecks occur in the network and how adding resources to the limiting queues can improve throughput and response time, while adding capacity to queues that are not busy provides little benefit. The results of the entire lab show that utilization, saturation, errors, response time, throughput, and topology must be considered together when modeling a queueing system.

REFERENCES

[1] B. Gregg, "Thinking methodically about performance," *Communications of the ACM*, vol. 56, no. 2, pp. 44–51, 2013. [Online]. Available: https://www.brendangregg.com/usemethod.html

[2] J. Leskovec and A. Krevl, "SNAP datasets: Texas road network," https://snap.stanford.edu/data/roadNet-TX.html, 2014.

[3] L. Kleinrock, *Queueing Systems, Volume I: Theory*. John Wiley & Sons, 1975. [Online]. Available: https://www.wiley.com/en-us/Queueing+Systems%2C+Volume+1%3A+Theory-p-9780471491101

[4] USENIX, "LISA17 - Queueing Theory in Practice: Performance Modeling for the Working Engineer," Nov. 2017. [Online]. Available: https://www.youtube.com/watch?v=Hda5tMrLJqc

[5] Mathemaniac, "Queuing theory and Poisson process," Jun. 2023. [Online]. Available: https://www.youtube.com/watch?v=rBIQmwaoZfs